

Manual de utilizare

Chatbot Agency Template

Ghid practic pentru a porni proiectul, a configura un client nou și a publica un bot pentru un site.

Ce document este acesta?

Este ghidul de lucru pentru ultima versiune construită în chat: backend Node/TypeScript + Express, configurare per client în folderul clients/, UI generic în public/ și integrare OpenAI.

Documentul este orientat pe folosire practică: cum pornești proiectul local, cum faci un client nou, cum faci demo, cum publici botul și ce fișiere editezi când vrei schimbări.

Versiune de lucru: manual operațional + documentație tehnică scurtă

1. Cum este organizat proiectul

În forma actuală, proiectul funcționează ca un template multi-client: ai o singură aplicație și câte un folder separat pentru fiecare client.

Element	Ce faci
src/server.ts	Punctul de intrare al aplicației. Pornește serverul și expune endpoint-urile API.
src/core/clientConfig.ts	Citește config-ul fiecărui client și listează clienții disponibili.
src/core/chat.ts	Construiește promptul și apelează OpenAI pentru răspunsuri.
clients/<clientId>/config.json	Setările vizibile ale clientului: nume, culoare, mesaj de întâmpinare, FAQ etc.
clients/<clientId>/prompts/system-prompt.txt	Instrucțiunile de comportament ale botului pentru acel client.
public/chat.html	Interfața generică de chat; se schimbă în funcție de ? clientId=...
public/admin.html	Dashboard simplu pentru listarea clienților și deschiderea chatului.
public/widget-demo.html	Pagină demo cu buton plutitor de chat, utilă în prezentări către clienți.
.env	Variabile locale, în special OPENAI_API_KEY.

Ideea centrală

Pentru un client nou nu rescrii aplicația. În majoritatea cazurilor doar creezi un folder nou în clients/, completezi config.json și system-prompt.txt, apoi folosești linkul generic /chat.html?clientId=nume-client.

2. Cum pornești proiectul local

- Deschizi proiectul în VS Code.
- Verifici să existe fișierul .env cu OPENAI_API_KEY.
- În terminal rulezi npm run dev.
- Testezi în browser sau cu curl endpoint-urile și paginile din public/.

```
npm install
```

```
npm run dev
```

Ce trebuie să vezi

În terminal: Server running on <http://localhost:3000>

În browser: </admin.html>, </chat.html?clientId=example-client>, </widget-demo.html?clientId=example-client>

Checklist rapid înainte să lucrezi la un bot

- node_modules există sau ai rulat npm install
- .env există și conține cheia OpenAI
- npm run dev pornește fără erori
- clientul pe care vrei să-l testezi are config.json valid
- dacă vrei ton personalizat, există și prompts/system-prompt.txt

3. Cum construiești un bot nou pentru un site

Fluxul normal de lucru este acesta:

- 1) primești informațiile clientului: tip business, întrebări frecvente, ton de voce, culori, link site;
- 2) creezi un folder nou în clients/;
- 3) copiezi structura de la example-client;
- 4) completezi config.json;
- 5) scrii system-prompt.txt;
- 6) testezi în </chat.html?clientId=...> și </widget-demo.html?clientId=...>;
- 7) ajustezi textele și comportamentul până când demo-ul este ok;
- 8) publici aplicația și folosești linkul sau iframe-ul pe site-ul clientului.

Pas cu pas

Pas	Acțiune
Pasul 1	Creezi folderul clients/nume-client/. Recomandat: numele folderului să fie același cu id-ul din config.json.
Pasul 2	Adaugi config.json. Aici setezi name, brandColor, openingMessage, description, toneOfVoice, faqButtons etc.
Pasul 3	Adaugi prompts/system-prompt.txt. Aici scrii cum trebuie să răspundă botul și ce să evite.
Pasul 4	Deschizi browserul la /chat.html?clientId=nume-client și verifici header, mesajul de întâmpinare și FAQ-urile.
Pasul 5	Deschizi /widget-demo.html?clientId=nume-client dacă vrei demo cu

Pas	Acțiune
	buton plutitor de chat.
Pasul 6	Dacă răspunsurile nu sunt bune, modifici system-prompt.txt și/sau config.json, apoi reîncarci pagina.
Pasul 7	Când totul este ok, trimiți clientului demo-ul sau integrezi widget-ul în site-ul lui.

Exemplu minimal de config.json

```
{
  "id": "petjungle",
  "name": "Pet Jungle",
  "brandColor": "#009688",
  "defaultLanguage": "ro",
  "description": "Magazin online de hrană și accesorii pentru animale.",
  "toneOfVoice": "prietenoasă, jucăușă, dar profesionistă",
  "openingMessage": "Bună! Sunt asistentul Pet Jungle. Cu ce te pot ajuta?",
  "allowedTopics": ["produse", "comenzi", "livrare", "retur"],
  "faqButtons": [
    "Ce tipuri de hrană recomandați pentru câini?",
    "Care sunt costurile și timpul de livrare?",
    "Cum pot returna o comandă?"
  ]
}
```

Exemplu minimal de system-prompt.txt

Ești un asistent virtual pentru Pet Jungle.
 Răspunzi prietenos, scurt și clar.
 Ajuți clienții cu produse, livrare și retur.
 Nu inventezi stoc, prețuri sau politici pe care nu le cunoști.
 Dacă nu știi ceva, spui asta sincer și recomanzi contactul uman.

4. Paginile importante și când le folosești

Pagină / endpoint	La ce folosește
/admin.html	Pentru tine. Vezi toți clienții disponibili și deschizi rapid chatul fiecăruia.
/chat.html?clientId=...	Pentru testarea directă a botului într-o fereastră standard.
/widget-demo.html?clientId=...	Pentru demo comercial: buton plutitor de chat peste o pagină generică.

Pagină / endpoint	La ce folosește
/api/clients	Pentru debugging sau integrare: listează clienții existenți.
/api/clients/:clientId	Returnează configul unui client.
/api/chat/:clientId	Primește un mesaj și întoarce răspunsul generat de model.

Cum faci un demo pentru client

- Pornești local aplicația.
- Deschizi /widget-demo.html?clientId=nume-client.
- Arăți butonul plutitor, deschizi chatul și pui 3-4 întrebări reale.
- Dacă clientul vrea alt text sau alt ton, modifici configul și promptul, apoi dai refresh.

5. Ce editezi când vrei schimbări

Element	Ce faci
Mesaj de întâmpinare	clients/<clientId>/config.json -> openingMessage
Culoare header	clients/<clientId>/config.json -> brandColor
Nume client	clients/<clientId>/config.json -> name
Tonul răspunsurilor	config.json -> toneOfVoice și prompts/system-prompt.txt
FAQ pe butoane	config.json -> faqButtons
Dimensiune / stil fereastră chat	public/chat.html și public/widget-demo.html
Butonul plutitor	public/widget-demo.html sau snippet-ul pus pe site-ul clientului
Reguli de business și limitări	prompts/system-prompt.txt

Regulă utilă

Dacă schimbarea este specifică unui singur client, încearcă mai întâi să o rezolvi din config.json sau prompt. Dacă schimbarea ține de interfață pentru toți clienții, modifici public/chat.html. Dacă doar un client vrea un UI complet diferit, poți crea un fișier nou în public/, de exemplu chat-client-special.html.

6. Cum îl publici și cum îl folosești pe un site

Fluxul de publicare poate fi pe orice hosting Node.js. Dacă folosești Railway sau un serviciu similar, pașii practici sunt aceiași la nivel de proiect:

- pui codul într-un repository Git;
- creezi un proiect nou pe platforma de hosting;
- conectezi repository-ul;
- setezi variabila OPENAI_API_KEY în environment variables;
- te asiguri că scriptul de start pornește aplicația Node;
- după deploy, primești un URL public pentru aplicație.

După publicare

- Dashboard public: <https://domeniu-tau/admin.html>
- Chat client: <https://domeniu-tau/chat.html?clientId=nume-client>
- Demo widget: <https://domeniu-tau/widget-demo.html?clientId=nume-client>

Integrare într-un site real

```
<div id="chat-widget-button" onclick="toggleChat()">☺ Chat</div>
<iframe id="chat-widget-iframe" src="https://domeniu-tau/chat.html?clientId=petjungle"></iframe>
<script>
  function toggleChat() {
    const frame = document.getElementById("chat-widget-iframe");
    frame.style.display = frame.style.display === "block" ? "none" : "block";
  }
</script>
```

Observație

Pentru demo nu ai nevoie să intri în site-ul clientului. Folosești widget-demo.html. Pentru implementare finală, clientul pune un snippet asemănător în site și iframe-ul încarcă chat.html?clientId=... de pe serverul tău.

7. Documentație tehnică scurtă

Flux tehnic al unui mesaj

- Userul scrie în public/chat.html.
- Frontend-ul face POST la /api/chat/:clientId cu body JSON { message }.
- server.ts primește requestul și apelează generateChatReply(clientId, message).
- chat.ts citește configul clientului și system-prompt.txt.
- chat.ts construiește system prompt-ul și trimite requestul către OpenAI.
- Răspunsul este trimis înapoi către browser și afișat în interfață.

Date de configurare pe client

Cheie	Rol
id	Identificator intern; ideal identic cu numele folderului.
name	Numele afișat în UI.
brandColor	Culoarea headerului în interfață.
defaultLanguage	Limba preferată pentru răspunsuri.
description	Descriere scurtă a businessului.
toneOfVoice	Tonul de voce dorit.
openingMessage	Primul mesaj afișat în chat.
allowedTopics	Subiecte permise / prioritare.
faqButtons	Întrebări rapide afișate ca butoane în UI.

Endpoint-uri

```
GET /          -> health check simplu
GET /api/clients -> lista tuturor clienților
GET /api/clients/:id -> config pentru un client
POST /api/chat/:id -> generează răspuns pentru un mesaj
GET /admin.html -> dashboard UI
GET /chat.html -> chat generic
GET /widget-demo.html -> demo cu buton plutitor
```

8. Probleme frecvente și soluții

Problemă	Soluție
Clientul nu apare în admin.html	Verifică dacă are folder propriu în clients/, dacă există config.json și dacă JSON-ul este valid.
/chat.html spune că nu găsește clientul	Verifică URL-ul: ?clientId=... trebuie să corespundă cu id-ul / folderul.
Botul nu răspunde	Verifică OPENAI_API_KEY și terminalul pentru erori.
Tonul nu este bun	Rescrie system-prompt.txt și toneOfVoice. Apoi testează din nou.
FAQ-urile nu se văd	Verifică să existe faqButtons în config.json și să fie un array valid.
UI-ul nu arată cum vrei	Modifică CSS-ul în public/chat.html sau public/widget-

Problemă	Soluție
	demo.html.
Curl nu merge în PowerShell	Folosește curl.exe sau Command Prompt / cmd.
npm dă eroare în PowerShell	Folosește Command Prompt sau schimbă Execution Policy.

9. Checklist operațional pentru un client nou

- Am creat folderul clients/<clientId>/
- Am completat config.json
- Am creat prompts/system-prompt.txt
- Am pornit proiectul cu npm run dev
- Am testat /chat.html?clientId=<clientId>
- Am testat /widget-demo.html?clientId=<clientId>
- Am ajustat tonul, FAQ-urile și mesajul de întâmpinare
- Am pregătit linkul public sau snippet-ul pentru site
- Am notat eventualele limite: ce poate și ce nu poate spune botul

Rezumat în o singură frază

Ca să construiești un bot nou pentru un site în această versiune a proiectului: pornești aplicația, creezi un folder nou în clients/, completezi configul și promptul, testezi în chat.html și widget-demo.html, apoi publici linkul sau iframe-ul pe site-ul clientului.